

Informe de Arquitectura e Implementación

Estimación de Velocidad Promedio por Ruta por Mes para el SITM-MIO

1. Introducción

El Sistema Integrado de Transporte Masivo de Occidente, SITM-MIO, genera diariamente grandes volúmenes de información operacional a partir de los datagramas enviados por los buses. Estos datagramas contienen datos relevantes para el análisis del servicio, entre ellos información de posición GPS, timestamp de ocurrencia e identificadores asociados a la operación del vehículo.

El presente proyecto se enfoca en el diseño, implementación y validación de una arquitectura de procesamiento para estimar la velocidad promedio de desplazamiento de las rutas del SITM-MIO por mes, a partir de un conjunto piloto de datos históricos. Esta estimación busca apoyar el análisis de desempeño del sistema y aportar información útil para mejorar la confiabilidad percibida por los usuarios frente a los tiempos de desplazamiento de las rutas.

Para abordar el problema, se desarrollaron tres versiones de la solución: una versión monolítica secuencial, una versión monolítica concurrente basada en ThreadPool y una versión distribuida basada en el patrón Master-Worker. Estas versiones permiten comparar el comportamiento del sistema frente al crecimiento del volumen de datos y evaluar el punto a partir del cual resulta conveniente distribuir el procesamiento.

2. Objetivo

Diseñar, implementar y validar experimentalmente una arquitectura de software capaz de calcular la velocidad promedio mensual por ruta del SITM-MIO, utilizando datos históricos de datagramas GPS del proyecto piloto. La solución debe permitir comparar el desempeño de una implementación monolítica, una implementación concurrente y una implementación distribuida, considerando los atributos de calidad de correctitud, performance y escalabilidad.

3. Alcance del requerimiento R7

El alcance principal del proyecto corresponde al requerimiento R7: estimar la velocidad promedio de todas las rutas por mes usando los datos históricos de eventos de posición GPS enviados por los buses en el año seleccionado para el piloto.

Para cumplir este requerimiento, la solución procesa los datagramas históricos, extrae los campos necesarios para reconstruir trayectorias de buses, calcula velocidades entre puntos GPS consecutivos y posteriormente agrega dichos resultados por ruta y por mes. La salida final se presenta como un archivo CSV con los promedios calculados, permitiendo su consulta y posterior análisis.

4. Drivers de arquitectura y atributos de calidad

4.1. Identificación y Priorización de Drivers de Arquitectura

El sistema tiene como propósito calcular la velocidad promedio por ruta y por mes a partir de los datagramas del piloto SITM-MIO, filtrando por las rutas activas definidas en lines-241-ActiveGT.csv. El contexto del problema involucra entre 2.5 y 3 millones de eventos diarios, con proyección de crecimiento de la flota de buses, lo que hace necesario que la arquitectura priorice no solo la correctitud del resultado sino también la capacidad de procesar grandes volúmenes de datos eficientemente.

Los drivers de arquitectura identificados y su orden de prioridad son:

Prioridad	Driver	Tipo
1	Correctitud	Atributo de calidad funcional
2	Performance (tiempo de respuesta)	Atributo de calidad no funcional
3	Escalabilidad	Atributo de calidad no funcional

Driver 1 - Correctitud

El sistema debe calcular de forma completa y correcta la velocidad promedio por ruta y por mes para todas las rutas activas del piloto, produciendo una salida consistente independientemente de la versión de la arquitectura utilizada (monolítica, concurrente o distribuida). Este driver es prioritario sobre los demás porque ninguna mejora de rendimiento tiene valor si los resultados calculados son incorrectos. No requiere escenario QAW.

Driver 2 - Performance (tiempo de respuesta)

El sistema debe reducir el tiempo total de procesamiento del archivo de datagramas conforme aumenta el volumen de datos. Este driver es crítico porque se exige comparar tiempos entre la solución monolítica y la distribuida, y porque el dataset de producción involucra millones de registros que el monolito no puede procesar de forma eficiente. La correctitud tiene prioridad sobre la performance: ninguna optimización de velocidad es válida si altera los resultados del cálculo.

Driver 3 - Escalabilidad

La arquitectura debe permitir aumentar la capacidad de procesamiento de forma controlada, ya sea incrementando el número de hilos en la versión concurrente o el número de nodos en la versión distribuida, manteniendo siempre la correctitud del resultado y evidenciando mejoras medibles en el tiempo de ejecución a medida que el dataset crece. La escalabilidad y la performance están relacionadas: escalar horizontalmente es el mecanismo principal para mejorar los tiempos de respuesta ante datasets de mayor volumen. Sin embargo, escalar introduce overhead de comunicación, serialización y sincronización, que puede perjudicar la performance para datasets pequeños.

4.2 Escenarios QAW

Escenario 1 - Performance

Campo	Descripción
Proyecto	Sistema de cálculo de velocidades promedio SITM-MIO

Escenario y contexto	El equipo de análisis del CCO necesita calcular la velocidad promedio por ruta y por mes para las 241 rutas activas definidas en lines-241-ActiveGT.csv, usando el dataset completo datagrams4Pilot.csv, que contiene aproximadamente 9 veces más registros que el subconjunto de prueba datagrams-MiniPilot.csv. La versión monolítica procesa los registros de forma secuencial en un único hilo sobre la JVM, recorriendo linealmente cada fila del CSV, calculando la velocidad instantánea como distancia entre datagramas consecutivos dividida entre el tiempo transcurrido, y acumulando el promedio por ruta y mes. Este procesamiento secuencial escala linealmente con el volumen de datos: al multiplicar por 9 el tamaño del dataset, el tiempo de ejecución se multiplica proporcionalmente, haciendo inviable el procesamiento dentro de una ventana operativa aceptable para el CCO.
Objetivo del negocio	Reducir el tiempo de procesamiento del cálculo de velocidades para poder operar con el dataset completo datagrams4Pilot.csv de manera eficiente, minimizando el tiempo de espera para obtener el reporte mensual y reduciendo costos operativos asociados al uso prolongado de infraestructura de cómputo.
QA's Relevantes	Performance, Tiempo de respuesta
Estímulo	Se lanza el procesamiento completo del archivo datagrams4Pilot.csv (todas las rutas activas de lines-241-ActiveGT.csv) desde el nodo de procesamiento central.
Fuente de Estímulo	El equipo de operación del CCO ejecuta el sistema de cálculo de velocidades para generar el reporte mensual de velocidades promedio por ruta.
Artefacto(s)	Subsistema de procesamiento de velocidades; Componente LectorCSV (lectura y parsing de datagrams4Pilot.csv); Componente ParticionadorCSV (división del dataset en subconjuntos independientes); ThreadPool de Workers de cálculo concurrente; Componente Calculadora (cálculo de velocidad promedio por partición); Componente AggregatorResultados (consolidación de resultados parciales en mapa final).

Entorno	Sistema en ejecución batch sobre servidor del laboratorio (x104m03), con el dataset completo datagrams4Pilot.csv cargado en disco local, bajo condiciones normales de carga sin procesos concurrentes externos que compitan por CPU o memoria.
Respuesta	El sistema carga datagrams4Pilot.csv, particiona el dataset en N subconjuntos (uno por hilo disponible), procesa cada partición en paralelo mediante el ThreadPool, consolida los resultados parciales en un único mapa ordenado por ruta y mes, y escribe el archivo CSV de salida con todas las velocidades promedio correctamente calculadas, produciendo resultados idénticos a los de la versión monolítica.
Medida de Respuesta	La versión concurrente con ThreadPool (N hilos igual al número de núcleos disponibles en x104m03) debe reducir el tiempo total de ejecución al menos un 30% respecto a la versión monolítica para el archivo datagrams4Pilot.csv, manteniendo exactamente los mismos valores calculados. El tiempo se mide desde el inicio de la lectura del CSV hasta la escritura completa del archivo de salida.
Preguntas Relevantes	¿A partir de qué tamaño de dataset (número de filas) la versión concurrente supera en rendimiento a la monolítica, dado el overhead de creación y coordinación del ThreadPool? ¿Cuántos hilos resultan óptimos para el hardware disponible en x104m03?
Otros Aspectos	Con datagrams-MiniPilot.csv (dataset pequeño) el overhead de crear y coordinar hilos supera los beneficios del paralelismo: la versión monolítica resulta más rápida (~25 ms vs ~40 ms con hilos). Con datagrams4Pilot.csv se espera que la ganancia de paralelismo supere ese overhead y la versión con ThreadPool sea significativamente más rápida.

Escenario 2 - Escalabilidad

Campo	Descripción
Proyecto	Sistema de cálculo de velocidades promedio SITM-MIO

Escenario y contexto	La organización necesita procesar el dataset de producción completo del SITM-MIO a partir del archivo datagrams4Pilot.csv, que representa datos históricos de hasta 1300 buses con proyección de crecimiento a 2000, generando entre 2.5 y 3 millones de eventos diarios. La arquitectura debe poder escalar horizontalmente agregando nodos de procesamiento para manejar ese volumen sin colapsar ni comprometer la correctitud de los resultados, superando la limitación de una sola máquina.
Objetivo del negocio	Garantizar que el sistema pueda crecer junto con el volumen de datos del SITM-MIO sin necesidad de rediseñar la arquitectura, aprovechando múltiples nodos de cómputo disponibles en el laboratorio (x104m03, x205m03, x206m03) para reducir tiempos de procesamiento y costos operativos.
QA's Relevantes	Escalabilidad, Performance
Estímulo	Se incrementa el número de Workers en la arquitectura distribuida Master-Worker de 1 a 2 a 3 nodos, procesando el archivo datagrams4Pilot.csv con las rutas activas de lines-241-ActiveGT.csv en cada configuración.
Fuente de Estímulo	El arquitecto del sistema agrega nodos de procesamiento adicionales en los servidores del laboratorio para mejorar el tiempo de respuesta ante el dataset completo del piloto.
Artefacto(s)	Componente Master (orquestador de particiones, nodo x104m03); Componente Worker (procesador parcial en nodo remoto, nodos x205m03 y x206m03); Componente ParticionadorCSV; Componente Agregador de Resultados parciales; Infraestructura de comunicación ICE sobre TCP/IP.
Entorno	Sistema desplegado en múltiples nodos del laboratorio (x104m03, x205m03, x206m03), comunicados por red local, con el archivo datagrams4Pilot.csv accesible desde el nodo Master, bajo condiciones normales de operación sin tráfico de red externo significativo.
Respuesta	El Master carga datagrams4Pilot.csv y lines-241-ActiveGT.csv, distribuye las particiones del dataset entre los Workers disponibles, cada Worker calcula sus resultados parciales de forma independiente sobre su partición, y el Master consolida todos los resultados en el

	archivo CSV final, manteniendo la correctitud e integridad del cálculo.
Medida de Respuesta	Al pasar de 1 a 2 Workers el tiempo de ejecución debe reducirse al menos un 30% para datagrams4Pilot.csv. Al pasar de 2 a 3 Workers debe haber una reducción adicional de al menos un 15%, manteniendo resultados idénticos a los de la versión monolítica. Se identifica el punto de inflexión cuando agregar un Worker adicional no reduce el tiempo más de un 5%, indicando que el overhead de serialización y comunicación ICE supera la ganancia de paralelismo.
Preguntas Relevantes	¿A partir de cuántos Workers el overhead de serialización y comunicación ICE por TCP/IP empieza a superar la ganancia de paralelismo en la red del laboratorio? ¿Cuál es el número óptimo de Workers para datagrams4Pilot.csv en la infraestructura disponible?
Otros Aspectos	Con datagrams-MiniPilot.csv el distribuido es el más lento (~173 ms) debido al overhead de sockets ICE y serialización. Con datagrams4Pilot.csv se espera que la ventaja del procesamiento distribuido supere ese overhead y los tiempos con múltiples Workers sean menores que el monolito.

4.3. Trade-offs entre Drivers

La arquitectura enfrenta un trade-off fundamental entre simplicidad operativa y capacidad de procesamiento, directamente derivado de las relaciones entre los drivers identificados.

La correctitud tiene prioridad absoluta sobre performance y escalabilidad: ninguna optimización de velocidad ni ningún esquema de distribución es válido si produce resultados incorrectos o distintos a los de la versión monolítica. Esto implica que toda decisión arquitectónica de distribución debe estar acompañada de un mecanismo de consolidación que garantice la integridad del resultado final.

Performance y escalabilidad están relacionadas pero no son equivalentes: la escalabilidad es el mecanismo arquitectónico para alcanzar la performance requerida cuando el volumen de datos supera la capacidad de procesamiento de un solo nodo. Sin embargo, escalar introduce overhead de comunicación por red, serialización de objetos y sincronización de resultados, lo que puede perjudicar la performance para datasets pequeños como datagrams-MiniPilot.csv donde el costo de coordinación supera la ganancia de paralelismo.

La versión monolítica es la más simple de implementar, desplegar y mantener, pero escala linealmente con el volumen de datos. La versión concurrente con hilos mejora significativamente la performance para datagrams4Pilot.csv sin aumentar la complejidad del despliegue, pero está limitada a los recursos de una sola máquina. La versión distribuida Master-Worker con ICE permite escalar horizontalmente hacia múltiples nodos (x104m03, x205m03, x206m03), satisfaciendo el driver de escalabilidad, pero introduce la mayor

5. Diseño de la solución

La solución implementada procesa los datagramas históricos del SITM-MIO para calcular la velocidad promedio mensual de cada ruta activa. El flujo inicia con la carga del catálogo de rutas activas desde el archivo lines-241-ActiveGT.csv. Posteriormente, se leen los datagramas del archivo de entrada y se transforman en puntos GPS normalizados.

Cada datagrama válido se convierte en un punto de trayectoria que contiene el identificador del bus, el identificador de ruta, la latitud, la longitud y el timestamp. Las coordenadas GPS se normalizan dividiendo los valores originales entre 10.000.000, debido a que en el archivo se encuentran almacenadas como enteros escalados. Una vez normalizados los datos, los puntos se agrupan por la combinación routeld + busld, de forma que cada grupo represente la trayectoria de un bus específico dentro de una ruta específica.

Después de construir las trayectorias, los puntos de cada grupo se ordenan cronológicamente. Con pares consecutivos de puntos GPS se calcula la distancia recorrida mediante la fórmula de Haversine y se obtiene el tiempo transcurrido entre ambos timestamps. La velocidad se calcula dividiendo la distancia recorrida entre el tiempo transcurrido. Para evitar que errores de GPS, registros duplicados o saltos anómalos afecten el promedio, se descartan velocidades inválidas, negativas, infinitas o superiores a 120 km/h.

Finalmente, las velocidades válidas se agregan por ruta y por mes usando una llave compuesta por routeld y YearMonth. El resultado final se escribe en un archivo CSV, permitiendo consultar la velocidad promedio mensual calculada para cada ruta activa presente en los datos procesados.

6. Patrones y estilos arquitectónicos

Para abordar el procesamiento masivo de datos de telemetría del SITM-MIO, compuesto por más de ocho millones de registros de datagramas, se seleccionó una arquitectura distribuida orientada a satisfacer los atributos de calidad priorizados durante el proceso de diseño: **performance (tiempo de respuesta)** y **escalabilidad**. La solución propuesta combina un estilo distribuido basado en middleware con patrones arquitectónicos y tácticos que permiten distribuir la carga computacional entre múltiples nodos de procesamiento.

Estilo Arquitectónico: Sistema Distribuido Basado en Objetos

La arquitectura implementada sigue un estilo distribuido basado en objetos soportado por el middleware ZeroC Ice. En este enfoque, la comunicación entre nodos no se realiza mediante sockets TCP gestionados manualmente ni mediante servicios REST tradicionales, sino a través de objetos remotos definidos mediante la especificación Slice.

La adopción de este estilo permite abstraer los detalles de comunicación, serialización y sincronización entre procesos distribuidos, proporcionando una interfaz fuertemente tipada para el intercambio de información. Adicionalmente, el middleware emplea protocolos binarios optimizados que reducen el overhead asociado a formatos textuales como JSON, contribuyendo directamente a la reducción de los tiempos de respuesta requeridos por el escenario de calidad.

Patrón Arquitectónico Principal: Master-Worker

El patrón principal seleccionado para la solución es Master-Worker. Bajo este esquema, un nodo central denominado Master coordina la ejecución global del sistema, mientras que un conjunto de nodos Worker ejecuta las tareas de procesamiento distribuidas.

El Master es responsable de cargar los archivos de entrada, fragmentar el conjunto de datos en unidades de trabajo independientes y distribuir dichas unidades entre los Workers disponibles. Cada Worker procesa de forma autónoma el bloque asignado y retorna resultados parciales que posteriormente son consolidados por el Master.

Desde la perspectiva de performance, este patrón permite paralelizar el cálculo de velocidades promedio evitando la concentración de toda la carga computacional en un único proceso. Desde la perspectiva de escalabilidad, la arquitectura

permite incorporar nuevos Workers sin modificar la lógica de negocio existente, aumentando proporcionalmente la capacidad de procesamiento disponible.

Variante Master-Worker: Separable Dependencies

Aunque la arquitectura se fundamenta en el patrón Master-Worker clásico, la implementación concreta adopta la variante conocida como Separable Dependencies.

Esta variante se caracteriza por eliminar dependencias compartidas entre las subtarefas mediante la partición de los datos y el procesamiento independiente de cada fragmento. En la solución desarrollada, los chunks generados por el Master contienen toda la información necesaria para ser procesados de manera aislada por cada Worker, evitando la necesidad de sincronización entre nodos durante la fase de cálculo.

Los resultados parciales producidos por cada Worker son posteriormente combinados por el Master mediante una etapa de consolidación, siguiendo el principio de composición de resultados característico de esta variante arquitectónica.

La adopción de Separable Dependencies reduce significativamente el acoplamiento entre componentes distribuidos y mejora la capacidad de escalamiento horizontal del sistema.

Para optimizar la distribución de trabajo se implementó una estrategia de segmentación dinámica basada en bloques de tamaño fijo de 100.000 registros.

En lugar de realizar una única partición estática del dataset, el Master divide continuamente la información en chunks independientes que son distribuidos utilizando un esquema Round-Robin entre los Workers disponibles.

Esta decisión arquitectónica aporta múltiples beneficios. En primer lugar, evita la generación de mensajes excesivamente grandes que podrían superar las restricciones del middleware o de la infraestructura de red. En segundo lugar, mejora el balanceo de carga al distribuir el trabajo de manera uniforme entre los nodos participantes. Finalmente, facilita la recuperación ante fallos y permite una utilización más eficiente de los recursos disponibles.

Patrón Thread Pool

A nivel interno, cada Worker incorpora un Thread Pool implementado mediante ThreadPoolExecutor.

La finalidad de este patrón es reutilizar hilos previamente creados y evitar el costo asociado a la creación y destrucción continua de threads del sistema operativo durante el procesamiento de millones de registros.

Cuando un Worker recibe un chunk, las tareas internas de análisis y agregación son distribuidas entre los hilos disponibles del pool, permitiendo aprovechar los múltiples núcleos de procesamiento de la máquina anfitriona.

La combinación del patrón Master-Worker a nivel distribuido y Thread Pool a nivel local constituye una estrategia jerárquica de paralelización que contribuye significativamente al cumplimiento de los atributos de calidad definidos para el proyecto.

Clasificación según la Taxonomía de Flynn

La solución propuesta puede clasificarse dentro de la categoría MIMD (Multiple Instruction, Multiple Data) de la taxonomía de Flynn.

En este modelo, múltiples procesadores ejecutan flujos de instrucciones independientes sobre conjuntos de datos distintos de manera simultánea. Cada Worker procesa un chunk diferente del dataset mientras ejecuta su propia lógica de cálculo, permitiendo explotar el paralelismo tanto a nivel de red como a nivel de hardware local.

Esta clasificación resulta coherente con la naturaleza distribuida de la solución, donde múltiples máquinas participan simultáneamente en la resolución del problema.

Modelo Global de Memoria: NORMA

A nivel de sistema distribuido, la arquitectura sigue el modelo NORMA (No Remote Memory Access).

Cada nodo participante dispone de su propio espacio de memoria física independiente y aislado. Como consecuencia, ningún Worker puede acceder directamente a la memoria de otro Worker ni a la memoria del Master.

Toda la transferencia de información se realiza mediante intercambio explícito de mensajes utilizando el middleware ZeroC Ice y mecanismos de comunicación asíncrona soportados por CompletableFuture.

Este modelo favorece el desacoplamiento entre nodos y elimina los problemas asociados a la sincronización de memoria compartida en entornos distribuidos.

Comunicación Asíncrona mediante Futures

La distribución de tareas se realiza mediante invocaciones remotas asíncronas soportadas por ZeroC Ice a través del método `calcularAsync`.

Este mecanismo implementa el paradigma de programación basado en Futures y CompletableFuture, permitiendo que el Master continúe enviando nuevas tareas mientras otras se encuentran en ejecución o tránsito por la red.

Gracias a esta estrategia, los tiempos de comunicación y los tiempos de procesamiento pueden solaparse, incrementando el grado de paralelismo efectivo del sistema y mejorando el tiempo total de ejecución.

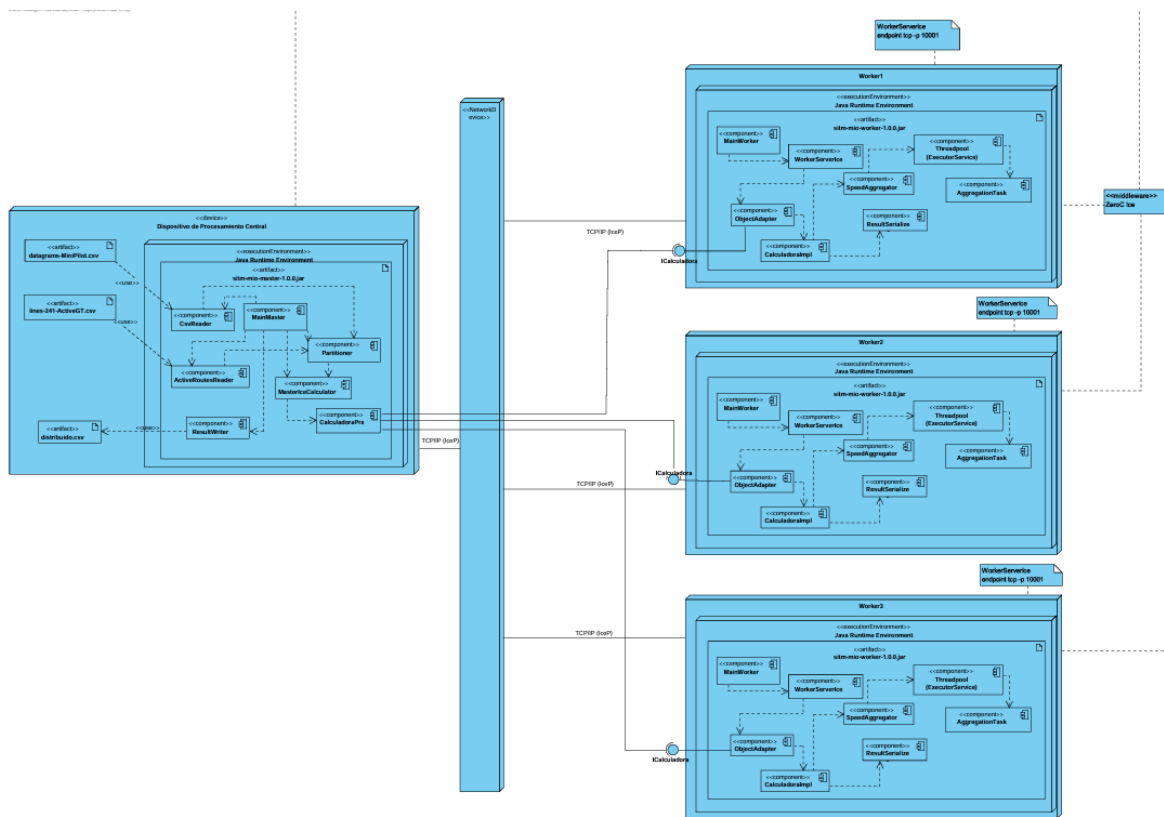
Soporte a los Atributos de Calidad

La arquitectura propuesta fue diseñada específicamente para satisfacer los atributos de calidad priorizados durante la evaluación arquitectónica.

Respecto al atributo de performance, la distribución de trabajo mediante chunks, la comunicación asíncrona, la ejecución paralela entre Workers y el uso de Thread Pools permiten reducir significativamente los tiempos de respuesta frente a una solución monolítica secuencial.

Respecto al atributo de escalabilidad, la independencia de los Workers, el modelo NORMA y la capacidad de incorporar nuevos nodos de procesamiento sin modificar la lógica central permiten incrementar progresivamente la capacidad computacional del sistema conforme aumenta el volumen de información procesada.

7. Diagrama de Deployment



Se puede observar de manera clara el patrón master-worker, al observar el flujo de los componentes, se evidencia el patrón separable dependencies y threadpool.

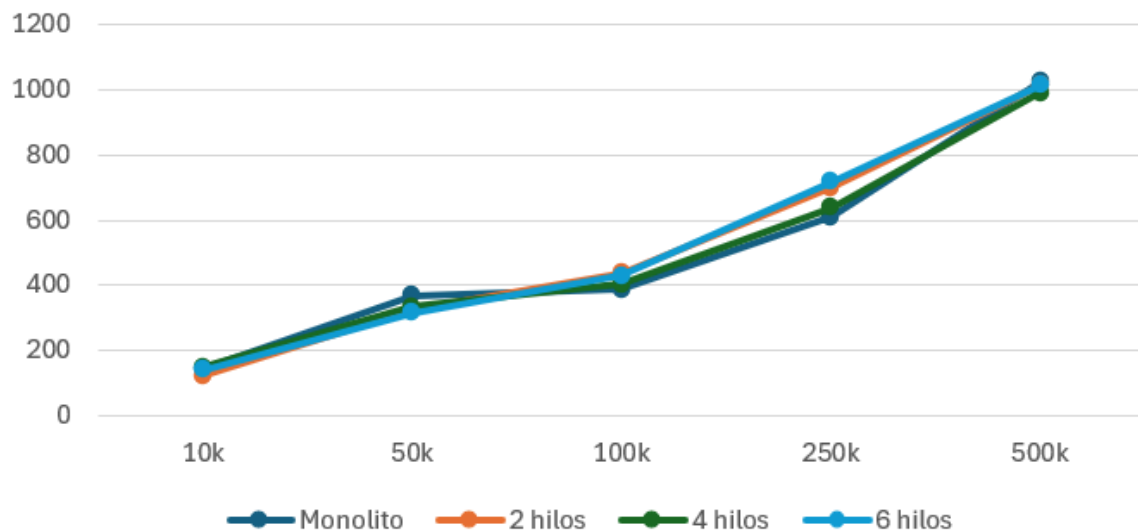
8. Implementación y validación experimental

Para validar experimentalmente la solución, se ejecutaron diferentes versiones del sistema sobre datasets de distintos tamaños. La versión monolítica se tomó como línea base funcional y de desempeño. Posteriormente, se ejecutó la versión concurrente con ThreadPool usando 2, 4 y 6 hilos, con el fin de comparar el impacto de la concurrencia sobre el tiempo de procesamiento.

Los datasets utilizados para la validación fueron subconjuntos de 10k, 50k, 100k, 250k y 500k registros, además del dataset FullMiniPilot. En todos los casos, la salida esperada corresponde a un archivo CSV con los resultados agregados por ruta y mes. Para el dataset FullMiniPilot.

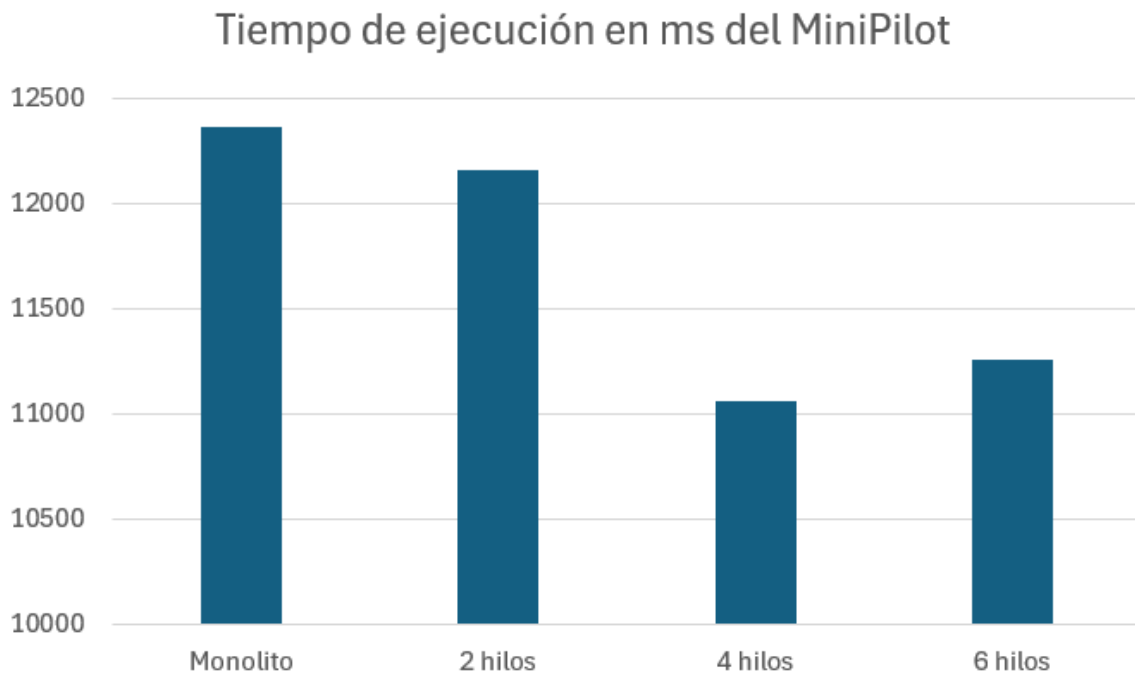
Dataset	Monolito	2 hilos	4 hilos	6 hilos
10k	146	124	150	141
50k	369	324	335	317
100k	388	439	405	430
250k	610	699	639	719
500k	1025	1015	991	1013

Comparación de tiempos (ms) de ejecución por tamaño de dataset



La Figura 1 muestra que, para datasets pequeños y medianos, los tiempos de ejecución de la versión monolítica y las versiones con hilos son similares. No se observa una mejora consistente al aumentar la cantidad de hilos, lo cual puede explicarse por el overhead asociado a la creación, planificación y sincronización de tareas. En estos tamaños de entrada, el costo adicional de coordinar el procesamiento concurrente puede compensar o incluso superar la ganancia obtenida por paralelización.

Dataset	Monolito	2 hilos	4 hilos	6 hilos
FullMiniPilot	12368	12159	11060	11256



En el dataset FullMiniPilot se observa una diferencia más clara. La versión monolítica obtuvo un tiempo de 12.368 ms, mientras que la versión con 4 hilos obtuvo el mejor resultado, con 11.060 ms. Esto representa una mejora moderada frente a la línea base monolítica. La versión con 6 hilos también mejoró el tiempo del monolito, aunque no superó a la configuración de 4 hilos. Este comportamiento sugiere que, para el ambiente de ejecución utilizado, 4 hilos ofrecen un mejor balance entre paralelismo y overhead de coordinación.

Estos resultados permiten concluir que la concurrencia no necesariamente mejora el desempeño en todos los tamaños de entrada. Para volúmenes pequeños, la versión monolítica puede ser suficiente e incluso competitiva. Sin embargo, a medida que el volumen de datos aumenta, la paralelización empieza a mostrar beneficios. Por esta razón, la versión distribuida se justifica principalmente para datasets de mayor escala, como datagrams4Pilot.csv o históricos completos, donde el volumen de datagramas puede superar la capacidad conveniente de procesamiento de un único proceso local.